# Diseño del algoritmo

TT 2027-A126 · Evidencia 2 del periodo 29 de septiembre – 12 de octubre de 2026

**Actividad del cronograma:** implementación del algoritmo bioinspirado: representación del individuo, operadores de cruce y mutación, y manejo de restricciones por dominancia restringida (Martínez).

**Algoritmo:** NSGA-II (Deb *et al.*, 2002) con dominancia restringida (Deb, 2000), sobre dos objetivos que se minimizan: $f_1$, la distancia media a los servicios, y $f_2$, el tiempo de evacuación.

**Código:** `experimentos/frente.py` (el ciclo, la dominancia, el ordenamiento y el cruce), `experimentos/potencial.py` (la mutación y la evaluación) y `nucleo/muestreo.py` (la población inicial). Hay una explicación paso a paso, con gráficas y una corrida real, en `cuadernos/multiobjetivo.ipynb`.

## 1. Representación del individuo

Un individuo es una **distribución**: la lista de módulos con su posición.

| Gen | Dominio | Por qué así |
|---|---|---|
| tipo | fijo: sanitario, comida, bebidas o primeros auxilios | cuántos hay de cada tipo lo decide el usuario, no el algoritmo |
| x, y | metros, en múltiplos de 0.25 m | precisión de replanteo en obra (`PASO_POS`) |
| rotación | 0 o 90° | los módulos son rectangulares |

En el caso de prueba (9 módulos: 3 sanitarios, 3 comidas, 2 bebidas y 1 de primeros auxilios) hay del orden de $10^{45}$ distribuciones distintas. Recorrerlas no es opción: hace falta una metaheurística.

**Identidad de un individuo.** Dos distribuciones que difieren en menos de una celda ocupan exactamente las mismas celdas y dan el mismo resultado. Por eso la llave del caché de evaluaciones es la **máscara de obstáculos rasterizada** y no las coordenadas.

## 2. Población inicial

Muestreo con rechazo, módulo por módulo (`muestreo.generar`, modo estricto):

- Se baraja el orden de los tipos, para que el orden de colocación no favorezca a ninguno.
- Cada módulo se propone al azar y se acepta solo si la distribución parcial sigue cumpliendo **todas** las restricciones.

El rechazo se usa **solo aquí**. Dentro del ciclo, dos padres válidos pueden dar un hijo inválido, y dos inválidos uno válido; tirarlos cortaría esa exploración.

## 3. Operadores

**Selección: torneo binario.** Se toman dos individuos al azar: gana el de mejor frente; si empatan, el de mayor distancia de apiñamiento.

**Cruce uniforme por módulo** (probabilidad 0.9). Cada módulo del hijo se hereda **entero** (x, y, rotación) de uno de los dos padres, con probabilidad ½.

- Los módulos se **emparejan por tipo**: el primer sanitario del hijo viene del primer sanitario de A o del de B.
- No se promedian coordenadas: el promedio de dos distribuciones distintas no se parece a ninguna y casi siempre deja módulos encimados.

> **Corrección del 13 de octubre.** El cruce emparejaba los módulos por su posición en la lista. Como el muestreo baraja el orden de los tipos, salían hijos con 4 comidas y 1 sanitario. En una corrida de prueba, con ese error el "mejor" llegaba a $f_1$ = 12.4 m, pero sin bebidas ni primeros auxilios; con el cruce corregido llega a 19.0 m. Los resultados del NSGA-II de investigación anteriores a esta fecha (`experimentos/OPTIMIZADOR.md`, secciones 0 a 2) hay que volver a correrlos.

**Mutación** (sin rechazo). Se modifican uno o dos módulos al azar:

- con probabilidad 0.20 se gira;
- con probabilidad 0.08 se manda a un lugar al azar del recinto (salto largo, para salir de óptimos locales);
- si no, se desplaza con un paso gaussiano de desviación σ, que va de 10 m en la primera generación a 2 m en la última: exploración al principio, ajuste fino al final.

**Reemplazo** $(\mu + \lambda)$**.** Padres e hijos compiten juntos. Se llena la nueva población frente por frente, y el último frente que no cabe completo se recorta por distancia de apiñamiento. Nadie se pierde sin que algo mejor ocupe su lugar.

## 4. Manejo de restricciones: dominancia restringida

Las seis restricciones duras se miden **en grado**: no solo si se cumplen, sino cuánto se violan.

| Restricción | Se mide en |
|---|---|
| G1 dentro del recinto | metros fuera |
| G2 sin encimarse | m² encimados |
| G3 sin pisar muros | m² sobre muros |
| G4 salidas despejadas | m² dentro de la zona de despeje |
| G5 área útil | m² perdidos por encima del 3 % |
| G6 módulos alcanzables | metros de frente que faltan |

La violación total es la suma. Al comparar dos individuos (Deb, 2000):

1. Un válido le gana a cualquier inválido, sin mirar los objetivos.
2. Entre dos inválidos, gana el que viola menos.
3. Entre dos válidos, la dominancia de Pareto de siempre.

**¿Por qué no penalizar?** Sumar la violación a los objetivos obliga a inventar un peso que mezcla metros cuadrados de violación con segundos de evacuación, y ese peso decide el resultado. Con la dominancia restringida no hay peso que elegir.

> **Corrección del 13 de octubre.** La zona ocupable (G5 y G6) erosionaba con un radio de 2 celdas, que en celdas de 1 m exige pasillos de 5 m en vez de 3 m. Con el radio corregido, sobre los 1,776 layouts del experimento de correlación, los válidos pasan de 1,206 (67.9 %) a 1,227 (69.1 %): 21 se vuelven válidos y ninguno deja de serlo.

## 5. Parámetros del algoritmo

| Parámetro | Valor | Nota |
|---|---|---|
| población | 48 | |
| generaciones | 50 | 2,448 evaluaciones por corrida |
| probabilidad de cruce | 0.9 | |
| módulos que muta cada hijo | 1 o 2 | |
| probabilidad de giro | 0.20 | |
| probabilidad de salto largo | 0.08 | |
| paso de mutación σ | de 10 m a 2 m | decae linealmente con las generaciones |

El costo de cada evaluación y el presupuesto que alcanza están en la evidencia 3 (`docs/ACOPLAMIENTO-Y-COSTO.md`).

## 6. Estado y siguiente paso

- **Listo:** representación, población inicial, operadores, dominancia restringida, ordenamiento no dominado y apiñamiento, en `experimentos/frente.py`.
- **Corregido hoy:** el cruce y la zona ocupable.
- **Siguiente periodo (13–26 de octubre):** volver a correr el NSGA-II con las correcciones, con repeticiones independientes para medir la variabilidad entre semillas, e integrarlo al prototipo, que hoy usa un optimizador $(\mu + \lambda)$ de un solo objetivo (`nucleo/optimizar.py`).

## Referencias


- Deb, K., Pratap, A., Agarwal, S. y Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2), 182–197. doi:10.1109/4235.996017
- Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2–4), 311–338. doi:10.1016/S0045-7825(99)00389-8